HOW THE ML WORKS, PART 1 OF 5

Foundations: from pixels to a prediction

Every model in Vellum starts from the same thing: an image is just a grid of numbers. This page builds up, step by step, how a neural network turns that grid into one of four answers. It covers the Module 7 topics of perception, image formation and object recognition.

---

1. An image is a grid of numbers

A digital image is made of pixels. Each pixel stores three numbers, the brightness of **red**, **green** and **blue** light, each from 0 (none) to 255 (full). So a colour image is really three grids stacked on top of each other. These grids are called **channels**.

Live: the document 01_certificate_course_completion.png, split into its three channels. Brighter means more of that colour.



Full colour | Red channel (0 to 255) | Green channel (0 to 255) | Blue channel (0 to 255)

In code this becomes a **tensor**: a block of numbers with a shape. Vellum resizes every page to 320 x 320 (padding with grey so the document keeps its shape), giving a tensor of shape **3 x 320 x 320 = 307,200 numbers**. Before the network sees them, each value is scaled to 0 to 1 and normalised with the ImageNet statistics the CNN was originally trained with:

```
x_normalised = (pixel / 255 - mean) / std
mean = [0.485, 0.456, 0.406]   std = [0.229, 0.224, 0.225]   (one value per R, G, B)
```

2. Machine learning and deep learning

Machine learning means the program learns its rules from examples instead of a person writing them. It is shown thousands of labelled documents and adjusts numbers (weights) until its answers match the labels.

Classic machine learning	A person decides what to measure (the features), the algorithm learns how to weigh them. In Vellum: the text model. A person chose "count words and letter groups", logistic regression learned which ones matter.
Deep learning	The network learns the features too, straight from the raw pixels, through many stacked layers. In Vellum: the EfficientNet-B0 CNN and CLIP. Nobody told them what a seal or a table looks like.

3. A neuron, and a network of them

A single artificial neuron multiplies each input by a weight, adds them up with a bias, and passes the sum through an activation function:

```
output = activation( w1*x1 + w2*x2 + ... + wn*xn + b )
ReLU(z) = max(0, z)        SiLU(z) = z * sigmoid(z)   (the one EfficientNet uses)
```

A layer is many neurons side by side; a network is many layers in a row. The last layer of Vellum's CNN has 4 neurons, one per bucket. Their raw scores go through **softmax**, which turns them into probabilities that add up to 1:

```
P(class i) = e^(score_i) / ( e^(score_1) + e^(score_2) + e^(score_3) + e^(score_4) )
```

Training repeats three steps thousands of times: predict, measure how wrong the prediction was with the cross-entropy loss (`loss = -log P(correct class)`), then nudge every weight a little in the direction that lowers the loss. Working out that direction for every weight at once is called **backpropagation**; the nudging is **gradient descent** (Vellum uses the AdamW variant).

4. Convolution: how a CNN looks at pixels

A fully connected layer on 307,200 inputs would need millions of weights per neuron and would ignore the fact that nearby pixels belong together. A **convolutional layer** instead slides a small grid of weights, a **kernel**, across the image. At each position it multiplies the 3 x 3 pixels under it by the 9 weights and adds them up. The result is a new grid called a **feature map**.

INPUT: A DARK STROKE ON PAPER

250	250	30	30	250	250
250	250	30	30	250	250
250	250	30	30	250	250
250	250	30	30	250	250
250	250	30	30	250	250
250	250	30	30	250	250

KERNEL: VERTICAL EDGE DETECTOR

-1	0	1
-1	0	1
-1	0	1

FEATURE MAP (4 X 4)

-660	-660	660	660
-660	-660	660	660
-660	-660	660	660
-660	-660	660	660

Worked out by this page in your browser. The amber 3 x 3 window gives (30+30+30) - (250+250+250) = -660: a strong negative where paper turns to ink, a strong positive where ink turns back to paper, and 0 where nothing changes. The kernel has found the two edges of the stroke.

In a real CNN the kernel weights are not written by hand: they are **learned** during training. Early layers end up detecting edges and corners, middle layers combine them into strokes, lines of text and table borders, and deep layers respond to whole layouts such as a certificate's border and seal. A layer runs many kernels at once, and each kernel produces one channel of the output. That is why the channel count grows (3, then 32, then up to 1280) while the image shrinks.

5. How 307,200 numbers become 4

A convolution with **stride 2** moves its kernel two pixels at a time, so its output is half as wide and half as tall. EfficientNet-B0 does this five times, so 320 pixels become 320 / 2^5 = **10**. Meanwhile the number of channels grows. So the network trades *where* things are for *what* they are. Finally, **global average pooling** takes the mean of each 10 x 10 channel, leaving one number per channel, and two dense layers map those 1280 numbers to the 4 buckets.

STEP	SHAPE (H X W X CHANNELS)	NUMBERS
Input image (letterboxed)	`320 x 320 x 3`	`307,200`
Stem convolution, stride 2	`160 x 160 x 32`	`819,200`
After stages 1 to 3	`40 x 40 x 40`	`64,000`
After stages 4 and 5	`20 x 20 x 112`	`44,800`
After stages 6 and 7	`10 x 10 x 320`	`32,000`
Final 1x1 convolution	`10 x 10 x 1280`	`128,000`
Global average pooling	`1280`	`1,280`
Dense layer + ReLU	`256`	`256`
Output layer + softmax	`4 (one per bucket)`	`4`

These shapes are read from the trained model, not from a textbook. The next tab shows what each stage actually produces.

HOW THE ML WORKS, PART 2 OF 5

Model 1: EfficientNet-B0, a convolutional neural network

The first model in the cascade, and the one that answers most documents. It is a CNN that looks only at the pixels. It is the fastest of the three (about 37 ms per page on the RTX 4060 laptop GPU) and it scored 100% on the 448 held-out test images.

---

1. Why this network

EfficientNet-B0 (Tan and Le, Google, 2019) is the smallest member of a family designed by scaling depth, width and input resolution together instead of making one of them huge. It reaches high accuracy with only about 4 million weights, small enough to train on a laptop GPU in minutes. We did not train it from scratch: we started from weights already trained on ImageNet (1.28 million photos, 1,000 classes), so it already knew edges, textures and shapes, and then **fine-tuned** every layer on our documents. This is called **transfer learning**.

Total parameters	`4,336,512`
Convolutional part	`4,007,548`
Our classification head	`328,964`
Input	320 x 320 RGB, letterboxed (resized keeping the aspect ratio, padded with grey)
Output	4 probabilities, one per bucket

2. The building block: MBConv

Every stage after the stem is a stack of **MBConv** (mobile inverted bottleneck) blocks. Each block does four things in order:

1. **Expand:** a 1 x 1 convolution multiplies the channel count by 6 (MBConv6), giving the block more room to work.
2. **Depthwise convolution:** a 3 x 3 or 5 x 5 kernel filters each channel on its own. This is far cheaper than a normal convolution, which mixes all channels at every position.
3. **Squeeze and excitation:** the block averages each channel to one number, passes those through a tiny two-layer network, and uses the result to scale each channel up or down. It learns which channels matter for this image.
4. **Project:** a 1 x 1 convolution shrinks the channels back down. If the shape did not change, the block's input is added to its output (a **skip connection**), which keeps training stable in deep networks.

3. Live: what each stage sees in your document

Below, the document is run through the real trained network. For every stage you see its output shape and the 6 channels (feature maps) that fired most strongly. Bright means high activation. Watch the resolution fall and the patterns become more abstract: early maps still look like the page, late maps are coarse 10 x 10 grids that encode "certificate-like layout here".



Input: 01_certificate_course_completion.png
`320 x 320 x 3 = 307,200 numbers`

Stem 3 x 3 convolution, stride 2, BatchNorm, SiLU `160 x 160 x 32 = 819,200 numbers, 928 weights`



ch 29 | ch 15 | ch 7 | ch 20 | ch 8 | ch 16

Stage 1 MBConv1, 3 x 3 depthwise `160 x 160 x 16 = 409,600 numbers, 1,448 weights`



ch 1 | ch 11 | ch 10 | ch 14 | ch 6 | ch 15

Stage 2 MBConv6, 3 x 3 depthwise, stride 2, 2 blocks

`80 x 80 x 24 = 153,600 numbers, 16,714 weights`



ch 18 | ch 23 | ch 11 | ch 0 | ch 12 | ch 14

Stage 3 MBConv6, 5 x 5 depthwise, stride 2, 2 blocks

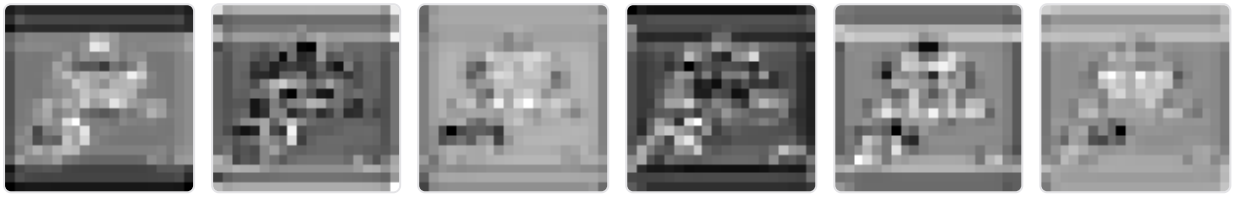
`40 x 40 x 40 = 64,000 numbers, 46,640 weights`



ch 37 ch 15 ch 30 ch 23 ch 27 ch 22

Stage 4 MBConv6, 3 x 3 depthwise, stride 2, 3 blocks

`20 x 20 x 80 = 32,000 numbers, 242,930 weights`



ch 45 ch 35 ch 47 ch 10 ch 70 ch 14

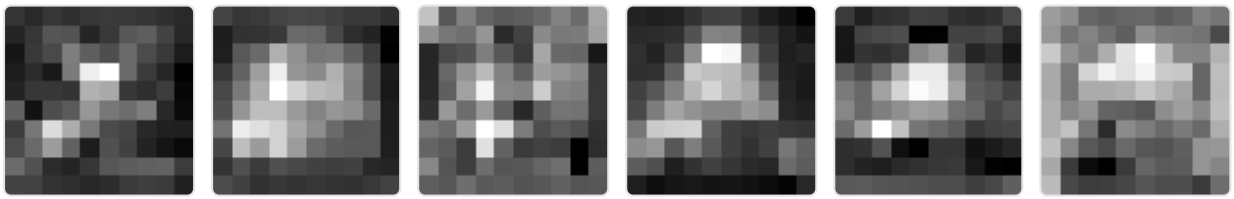
Stage 5 MBConv6, 5 x 5 depthwise, 3 blocks `20 x 20 x 112 = 44,800 numbers, 543,148 weights`



ch 15 ch 94 ch 50 ch 86 ch 29 ch 61

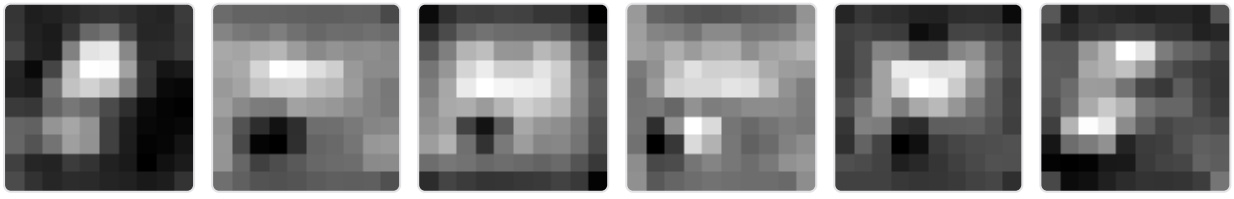
Stage 6 MBConv6, 5 x 5 depthwise, stride 2, 4 blocks

`10 x 10 x 192 = 19,200 numbers, 2,026,348 weights`



ch 157 ch 9 ch 167 ch 59 ch 181 ch 158

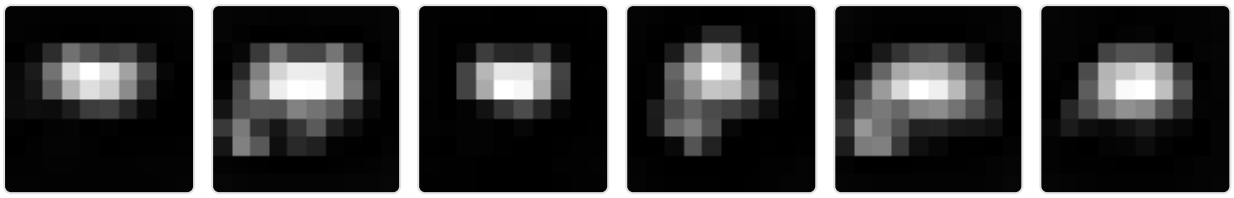
Stage 7 MBConv6, 3 x 3 depthwise `10 x 10 x 320 = 32,000 numbers, 717,232 weights`



ch 257 ch 310 ch 251 ch 113 ch 21 ch 236

Head conv 1 x 1 convolution to 1280 channels, BatchNorm, SiLU

`10 x 10 x 1280 = 128,000 numbers, 412,160 weights`



ch 447 ch 1107 ch 903 ch 463 ch 1093 ch 1232

Global average pooling, then the head

Each of the 1280 final 10 x 10 maps is averaged to a single number, giving a vector of 1280. The strongest 12 for this page:

`#447: 1.593` `#1107: 1.52` `#903: 1.464` `#463: 1.443` `#1093: 1.415` `#1232: 1.41`
`#952: 1.39` `#162: 1.378` `#1026: 1.367` `#364: 1.363` `#823: 1.359` `#1132: 1.291`

```
1280  ->  Linear(1280, 256) + ReLU  ->  Dropout(0.3)  ->  Linear(256, 4)  ->  softmax
```

Certificate	96.5%
Academic record	0.9%
Unrelated document	1.2%
Not a document	1.4%

4. How it was trained

Data	1,944 training, 407 validation, 448 test images: generated certificates, grade sheets, letters and LORs, plus RVL-CDIP and FUNSD scanned documents and Flickr and Imagenette photos
Augmentation	Random rotation up to 4 degrees, shifts, zoom 0.9 to 1.08, colour jitter, 15% turned grey. So the model does not memorise exact pixels.
Loss	Cross-entropy, weighted by class frequency, label smoothing 0.05
Optimiser	AdamW, learning rate 0.0006 with a one-cycle schedule, weight decay 0.0001
Schedule	12 epochs, batch size 32, mixed precision on the GPU, about 13 s per epoch
Result	100% on the 448 test images (all four classes)

5. Its weakness, and why it does not work alone

On 6 genuine scans from Wikimedia Commons that look nothing like the training data (a 1969 PhD diploma, a Spanish diploma, a handwritten record book), it got only **1 of 6** right. It learned the look of our training documents too well. But it was never more than 89% sure on those mistakes, while every correct test answer was at least 91.6% sure. So the cascade only trusts it at 90% or above, and passes everything else to CLIP.

Model 2: CLIP ViT-B/32, a vision-language model

The second model in the cascade. It runs only when the CNN is less than 90% sure, which in practice means real-world scans that look unlike our training data. CLIP was **never trained on our documents at all**. It classifies by comparing the image with sentences describing each bucket. This is called **zero-shot** classification.

---

1. What CLIP is

CLIP (Contrastive Language-Image Pre-training, OpenAI 2021) is two networks trained together: one turns an image into a list of 512 numbers, the other turns a sentence into a list of 512 numbers. The weights we use were trained by LAION on about 2 billion image and caption pairs from the internet. Training pulled the vectors of each matching image and caption together and pushed non-matching ones apart, so **an image and a sentence that describe the same thing end up pointing in the same direction**.

Total parameters	`151,277,313`
Image encoder	`87,456,000 (Vision Transformer, ViT-B/32)`
Text encoder	`63,165,952 (Transformer)`
Shared embedding size	`512`
Trained on our data	No. Zero-shot.

2. The image side: a Vision Transformer

A Vision Transformer has no convolutions. It cuts the image into square patches and treats each patch like a word in a sentence.



Live: 01_certificate_course_completion.png, cropped to 224 x 224 and cut into 49 patches.

1. The image is resized so its short side is 224 pixels and the centre 224 x 224 is kept.
2. It is cut into a 7 x 7 grid of **49 patches**, each 32 x 32 pixels x 3 channels = 3,072 numbers ("/32" in ViT-B/32).
3. Each patch is flattened and multiplied by a learned matrix to become a 768-number vector, a **token**.
4. A learned position vector is added to each token so the model knows where the patch was, and one extra **CLS** token is put in front: 50 tokens.
5. **12 transformer layers** follow. In each, every token looks at every other token through **self-attention** (12 heads), so a patch with a seal can learn that a patch with a signature line sits below it.
6. The CLS token's final vector is projected to 512 numbers. That is the image's embedding.

```
Attention(Q, K, V) = softmax( Q K^T / sqrt(d) ) V
Q, K, V: three learned projections of the tokens.  Each token gets a weighted mix of all the
others.
```

3. The text side: describing each bucket in words

Instead of training a classifier, we wrote 7 or 8 plain-English descriptions per bucket. Each is split into sub-word tokens (a vocabulary of 49,408, at most 77 tokens), run through the 12-layer text transformer, and turned into a 512-number vector. The vectors of one bucket are averaged into a single vector for that bucket (**prompt ensembling**), which is more robust than any single sentence. These are computed once when the server starts.

Certificate

- "a scan of a university degree certificate"
- "a photo of a diploma"
- "a certificate of completion with a signature and a seal"
- "an award certificate with a decorative border"
- "an academic diploma written in latin or another language"
- "a framed graduation diploma"
- "a course completion certificate"

Academic record

- "a university grade sheet with a table of courses and grades"
- "a student mark sheet with subjects and marks"
- "an academic transcript of records"
- "a report card with grades"
- "a student record book with handwritten grades"
- "a letter of recommendation from a professor on university letterhead"
- "a letter from a university registrar"

Unrelated document

- "a scanned invoice or receipt"
- "a filled in paper form"
- "a newspaper page"
- "a business memo or email printout"
- "a bank statement"
- "a restaurant menu"
- "a page of a scientific paper"
- "an advertisement flyer"

Not a document

- "a photo of a person"
- "a photo of an animal"
- "a photo of a landscape"
- "a photo of a street or a city"
- "a photo of food"
- "a photo of an object"
- "a selfie"
- "a photo of a vehicle"

4. Making the decision

Both kinds of vector are scaled to length 1, so their dot product is the **cosine similarity**: 1 when they point the same way, 0 when unrelated. The image is compared with the four bucket vectors, the similarities are multiplied by CLIP's learned temperature of 100, and softmax turns them into probabilities.

```
similarity_k = (image_vector . bucket_vector_k)          both of length 1
P(bucket k)  = softmax( 100 x similarity_k )
```

5. Strengths and weaknesses

Real-world scans	6 of 6 correct, including a 1969 diploma, a Spanish diploma and a handwritten record book
Our 448 test images	87.1%. It sometimes confuses our generated layouts, and can be wrong while 99% sure.
Speed	About 37 ms per page on the GPU. The first request after the server starts is slow (about 20 s measured) while the weights load.
Role in Vellum	The backup for the CNN. It is only asked when the CNN is unsure, which is exactly where it is strong.

Model 3: OCR and the TF-IDF text model

The third model classifies a document from the **words** on it rather than its pixels. To get the words, the page first goes through OCR (optical character recognition). The same OCR output is also what the field extractor reads to pull out the name, register number and CGPA, so OCR runs for every relevant document even when this model is not needed.

---

1. OCR: from pixels to text

Vellum uses RapidOCR, which runs Baidu's PP-OCRv4 models through ONNX Runtime on the CPU. It is three neural networks in a row:

1. **Text detection.** A convolutional network (DBNet style) outputs, for every pixel, the probability that it is part of text. Pixels above 0.3 are grouped into regions and each region becomes a box around one line of text.
2. **Direction check.** A small classifier looks at each line crop (48 x 192 pixels) and decides whether it is upright or upside down (0 or 180 degrees), and flips it if needed.
3. **Text recognition.** Each line is resized to 48 pixels tall and read left to right by a recognition network that outputs a probability for every character at every horizontal position. **CTC decoding** then merges repeated characters and removes blanks to give the final string, with a confidence score.

Output	Every text line, its box on the page and its confidence. These are the green boxes on the result page.
Speed	4 to 15 s per page measured on the 6 real scans. By far the slowest step, which is why it runs only when needed.
Weak on	Handwriting, very small print, heavy blur

2. TF-IDF: turning text into numbers

A classifier needs numbers, not words. TF-IDF gives each word a score that is high when the word appears often in *this* document but rarely across all documents. So "certificate" on a certificate scores high, while "the" scores near zero because it is everywhere.

```
tf-idf(t, d) = (1 + log tf(t, d)) x idf(t)          tf: count of term t in document d
idf(t)       = log( (1 + N) / (1 + df(t)) ) + 1     N: documents, df: documents containing t
then the whole vector is scaled to length 1
```

Two vocabularies are learned from the training text and joined into one vector of **32,597** numbers per document:

Word features	5,517 single words and two-word phrases ("grade sheet"), kept if they appear in at least 2 documents
Character features	27,080 groups of 3 to 5 letters ("cert", "grad"). These still match when OCR misreads one letter of a word.

3. Logistic regression: the classifier

Multinomial logistic regression learns one weight per feature per bucket: 4 x 32,597 weights. For a document it adds up weight x feature for each bucket and softmax turns the four sums into probabilities. A large positive weight means "this word is evidence for this bucket".

```
score_k = w_k . x + b_k        x: the 32,597 TF-IDF numbers
P(bucket k) = softmax(score)_k
```

STRONGEST WORD EVIDENCE THE MODEL LEARNED

Certificate	certificate
Academic record	pass, grade, student, programme

It was trained with class balancing (so a class with fewer examples counts as much) and a regularisation strength of C = 4. It is used only if OCR finds at least 5 words, so it never runs on a photo with no text.

4. How well it works, honestly

Test images with text (362)	96.7% correct
Real-world scans (6)	0 of 6. Their text (old print, handwriting, a Spanish diploma) looks nothing like the text of our training documents.
Role in Vellum	The last resort: it is reached only if both image models are under 90% sure. In our evaluation that never happened.

HOW THE ML WORKS, PART 5 OF 5

How one model is picked: the early-exit cascade

Vellum has three models but returns one answer from one model. It asks them one at a time, fastest first, and stops as soon as one is confident. This pattern is called an **early-exit cascade**, the same idea as the classic Viola-Jones face detector: cheap tests first, expensive ones only when needed.

1. The flow

TRY 1

EfficientNet-B0 CNN

Fastest, and 100% on the test images

~37 ms

Accept if at least 90% sure: answer and stop

↓ under 90% sure: ask the next model

TRY 2

CLIP ViT-B/32

Strong on real-world scans the CNN has never seen

~37 ms

Accept if at least 90% sure: answer and stop

↓ under 90% sure: ask the next model

TRY 3

OCR + text model

Slowest, because OCR must run first

4 to 15 s

Always accept (last resort): answer and stop

If no model reaches its threshold, the most confident answer seen is returned and marked as low confidence on the result page.

2. Why the threshold is 90%

The threshold was measured, not guessed. "Best" needs a way to judge quality without knowing the right answer, and the model's own confidence (its top softmax probability) is that signal, as long as a confident answer is a correct one. We checked exactly that on held-out data:

CNN, correct answers on 448 test images	lowest confidence 91.6%
CNN, wrong answers on the real scans	highest confidence 88.9%
So a threshold of 90%	keeps every correct CNN answer and rejects every wrong one, which then go to CLIP
CLIP on the real scans	every answer correct and at least 94.7% sure

```
for model in [CNN, CLIP, text]:
    p = model(page)
    if max(p) >= threshold[model]:
        return model, argmax(p)          # the remaining models never run
return the most confident answer seen
```

3. Measured result

448 test images	448 correct (100%). All answered by the CNN, CLIP never needed.
6 real-world scans	6 correct (100%). 1 answered by the CNN, 5 passed on to CLIP.
Text model	Never needed in either set
A rejected photo	About 40 ms end to end, because OCR is skipped when the page is not a credential

Source: `python -m ml.document_classifier.evaluate`, saved in models/document_classifier_metrics.json. This is the same accuracy as the earlier version that ran all three models and averaged them, at a fraction of the work. Caveat: the real-world set is only 6 scans, so the 90% rule should be re-checked as more real documents are added.

4. Against ma'am's requirements

Support multiple file types	PNG, JPG, WEBP, BMP, TIFF, GIF and PDF (first 5 pages). Anything else is refused with a message.
Process files using three models	All three are available and wired in: CNN, CLIP and the text model.
Return only the best and fastest result	Models are ordered fastest first. The first one confident enough answers.
Do not compare all three models	The result page shows one model's answer only. The others' outputs are never shown, and usually never computed.
Stop once a suitable result is available	The cascade returns the moment a model clears its threshold. Later models do not run at all.

5. Your last run

File	01_certificate_course_completion.png
Answered by	EfficientNet-B0 CNN
Confidence	96.5% (threshold 90%)
Models that ran	1
Time to decide	121 ms